

Session 3 — Applied Cryptography for Blockchain

Buổi 3 — Mật mã ứng dụng cho Blockchain

Blockchain Technology and Cryptocurrency

TS. Nguyễn Trung Thành

Hash • Merkle tree • Digital signatures • BIP39/32/44 • Addresses

Objectives / Mục tiêu

1. Hash functions: 3 security properties → where each lives in a blockchain
Hàm băm: 3 tính chất an toàn → mỗi tính chất nằm ở đâu trong blockchain
2. Merkle trees: build, prove, verify — why $O(\log n)$
Cây Merkle: dựng, chứng minh, kiểm — vì sao $O(\log n)$
3. ECDSA on secp256k1, the nonce k , Ethereum's `ecrecover`
ECDSA trên secp256k1, nonce k , `ecrecover` của Ethereum
4. Entropy → mnemonic → HD tree → address (the Session-1 wallet, formalized)
Entropy → mnemonic → cây HD → địa chỉ (chính là ví của Buổi 1, nay chặt chẽ)

Agenda — 3 hours / 3 giờ

Time	Content / Nội dung
0:00–0:55	Hash & Merkle — hàm băm & cây Merkle
0:55–1:05	Break / <i>giải lao</i>
1:05–2:00	Signatures, keys, addresses — chữ ký, khóa, địa chỉ
2:00–2:10	Break / <i>giải lao</i>
2:10–3:05	Lab: Merkle + sign/verify/recover

Recap / Ôn lại (Session 2)

- Each block stores the **hash** of the previous block
*Mỗi block chứa **giá trị băm** của block trước*
- Your wallet **signed** transactions in Session 1
*Ví của bạn đã **ký** giao dịch ở Buổi 1*
- Backup = 12 words
Sao lưu = 12 từ

Today we open these black boxes.

Hôm nay ta mở các hộp đen đó.

Hour 1 / Giờ 1

Hash functions & Merkle trees

Hàm băm & cây Merkle

Hash function / Hàm băm

H: bytes (any length) → 256 bits (fixed)

H: byte (độ dài bất kỳ) → 256 bit (cố định)

- **Deterministic** — same input, same digest / *tất định — cùng đầu vào, cùng giá trị băm*
- **Fast forward** — cheap to compute $H(m)$ / *tính xuôi rẻ*
- **Fixed size** — 1 byte or 1 GB → still 256 bits / *đầu ra luôn 256 bit*
- **Avalanche** — 1 bit flips ~half the output / *đổi 1 bit → đổi ~nửa đầu ra*

A **cryptographic** hash must also resist attackers → next slide.

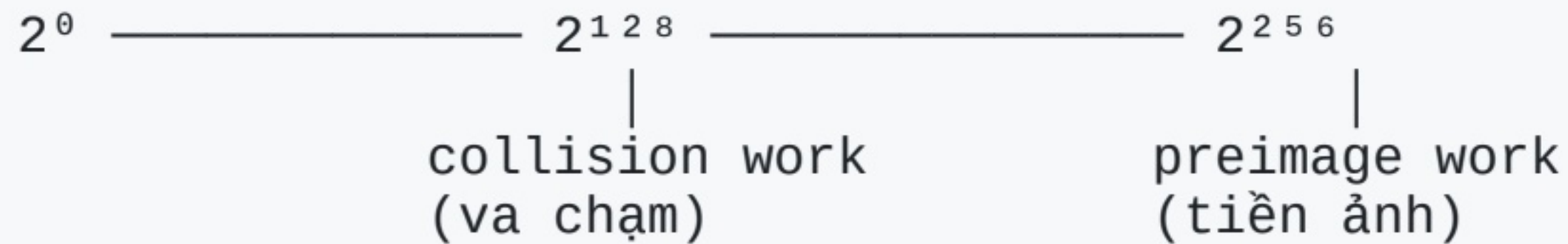
Băm **mật mã** còn phải kháng kẻ tấn công → trang sau.

Three security properties / Ba tính chất an toàn

#	Property / Tính chất	Meaning / Ý nghĩa
1	Preimage resistance / <i>kháng tiền ảnh</i>	given h , can't find m : $H(m)=h$
2	2nd-preimage / <i>kháng tiền ảnh thứ 2</i>	given m_1 , can't find $m_2 \neq m_1$ same hash
3	Collision resistance / <i>kháng va chạm</i>	can't find ANY colliding pair

- Difficulty: preimage $\approx 2^{256}$, collision $\approx 2^{128}$ (birthday)
Độ khó: tiền ảnh $\approx 2^{256}$, va chạm $\approx 2^{128}$ (birthday)
- Collision is the **weakest** link $\rightarrow$ it sets the security level
Va chạm là mắt xích yếu nhất $\rightarrow$ quyết định mức an toàn

Birthday bound — the number line / Cộn birthday



- $2^{128} \approx 3.4 \times 10^{38}$ tries / *phép thử*
- Even at $\sim 10^{18}$ hashes/s (whole Bitcoin network) $\rightarrow \sim \mathbf{10^{13}}$ **years**
Ngay ở $\sim 10^{18}$ băm/giây (toàn mạng Bitcoin) $\rightarrow \sim 10^{13}$ năm
- Age of the universe $\approx \mathbf{1.4 \times 10^{10}}$ **years** $\rightarrow$ collisions are *infeasible*, not impossible
Tuổi vũ trụ $\approx 1.4 \times 10^{10}$ năm $\rightarrow$ tìm va chạm bất khả thi trên thực tế

Demo — avalanche effect / Hiệu ứng tuyết lở

```
$ python3 -c "import hashlib; print(hashlib.sha256(b'Blockchain 2026').hexdigest())"  
73a4b2...e91f  
$ python3 -c "import hashlib; print(hashlib.sha256(b'blockchain 2026').hexdigest())"  
f0c81d...207a
```

One flipped letter → completely different digest

Đổi một chữ cái → giá trị băm khác hoàn toàn

No partial match, no "getting warmer" — output is unpredictable from input.

Không khớp một phần, không "nóng dần" — không đoán được đầu ra từ đầu vào.

Hash families — mind the trap / Các họ hàm băm — cẩn thận bẫy

Family / Họ	Used by / Dùng bởi
SHA-256 (SHA-2)	Bitcoin (double SHA-256), TLS certs
Keccak-256	Ethereum — <code>keccak256(...)</code> in Solidity
SHA3-256 (NIST)	standards; ≠ Keccak-256 (padding differs)
RIPEMD-160	Bitcoin addresses (160-bit)

Trap / Bẫy: Ethereum's `keccak256` is **not** NIST SHA3-256 — classic interview trap.

`keccak256` của Ethereum **không** phải SHA3-256 chuẩn NIST — bẫy phỏng vấn kinh điển.

Where hashes live / Băm nằm ở đâu

Use / Ứng dụng	Property / Tính chất
Block linking (prev-hash) / <i>liên kết block</i>	2nd-preimage + collision
Transaction ID / <i>mã giao dịch</i>	collision
PoW: $H(\text{block} \parallel \text{nonce}) < \text{target}$	preimage $\rightarrow$ brute force only
Address from pubkey / <i>địa chỉ từ khóa công khai</i>	preimage
Commit-reveal / <i>cam kết</i>	hiding + binding

One primitive, five jobs. *Một nguyên thủy, năm vai trò.*

Block linking / Liên kết block

Liên kết block bằng giá trị băm — Block linking by hash

Sửa 1 byte ở block cũ $\Rightarrow$ mọi prev-hash phía sau gãy • Change 1 byte $\Rightarrow$ every later prev-hash breaks



Băm cho KHẢ NĂNG PHÁT HIỆN sửa đổi (tamper-evidence); PoW/PoS thêm CHI PHÍ chống sửa đổi. Hashing gives tamper-EVIDENCE; PoW/PoS adds the COST of tampering. Kẻ tấn công phải đào lại toàn bộ block kế tiếp trong khi chuỗi trung thực vẫn dài thêm — the attacker must re-mine every successor while the honest chain keeps growing.

Tamper-evidence vs tamper-resistance / Phát hiện vs chống sửa

- Change 1 byte in block k $\rightarrow$ its hash changes $\rightarrow$ block $k+1$'s stored prev-hash no longer matches

Sửa 1 byte ở block k $\rightarrow$ băm đổi $\rightarrow$ prev-hash trong block $k+1$ không còn khớp

- Attacker must **re-mine every successor** while the honest chain keeps growing
Kẻ tấn công phải đào lại mọi block sau, trong khi chuỗi trung thực vẫn dài thêm

Hashing gives / Băm cho	Consensus adds / Đồng thuận thêm
tamper- evidence (detect) / <i>phát hiện</i>	tamper- resistance cost / <i>chi phí chống sửa</i>

"Computational amber" that fossilizes history / "Hố phách tính toán" hóa thạch lịch sử.

Think-pair-share (3')

If SHA-256 collisions became easy tomorrow, name two concrete attacks on Bitcoin.

Nếu ngày mai tìm va chạm SHA-256 trở nên dễ, nêu hai tấn công cụ thể vào Bitcoin.

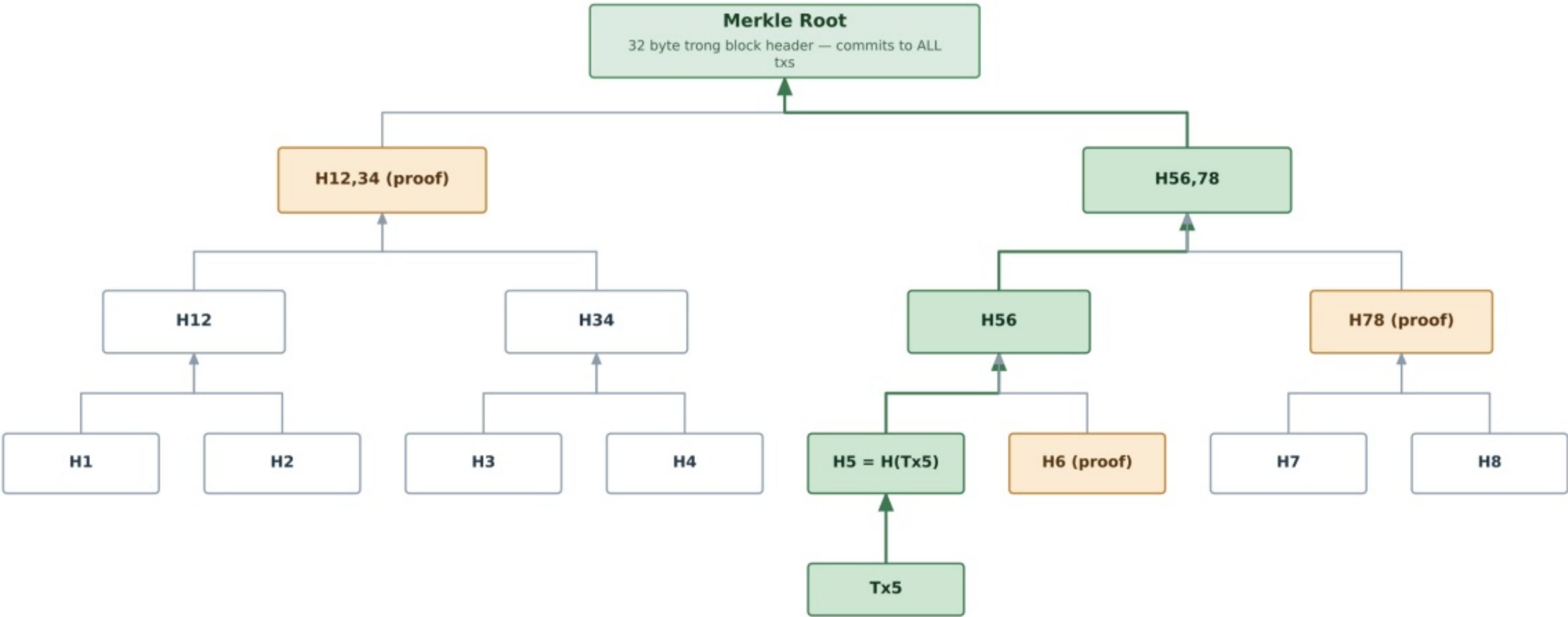
Hint / Gợi ý: two txs sharing one txid • a fake block with the same header hash.

Hai giao dịch trùng txid • một block giả có cùng băm header.

Merkle tree / Cây Merkle

Cây Merkle và bằng chứng cho Tx5 — Merkle tree & proof for Tx5

Xanh lá = đường tính lại của người kiểm • Cam = 3 băm trong proof = $\log_2(8)$ — Green = verifier recompute path • Orange = the 3 proof hashes



Proof(Tx5) = [H6, H78, H12,34] → chỉ $\log_2(n)$ băm: 1.024 giao dịch cần 10 băm = 320 byte thay vì cả block. Only $\log_2(n)$ hashes: 1,024 txs need just 10 hashes = 320 bytes instead of the whole block.

Building a Merkle tree / Dựng cây Merkle

- **Leaves** = $H(tx_1), H(tx_2), \dots$ / Lá = băm từng giao dịch
- **Parent** = $H(\text{left} \parallel \text{right})$ / nút cha = băm của (trái nối phải)
- Odd node at a level → **duplicate it** (Bitcoin rule) / nút lẻ → nhân đôi (quy tắc Bitcoin)
- Repeat up to a single **Merkle root** (32 bytes) / lặp lên tới một gốc Merkle (32 byte)

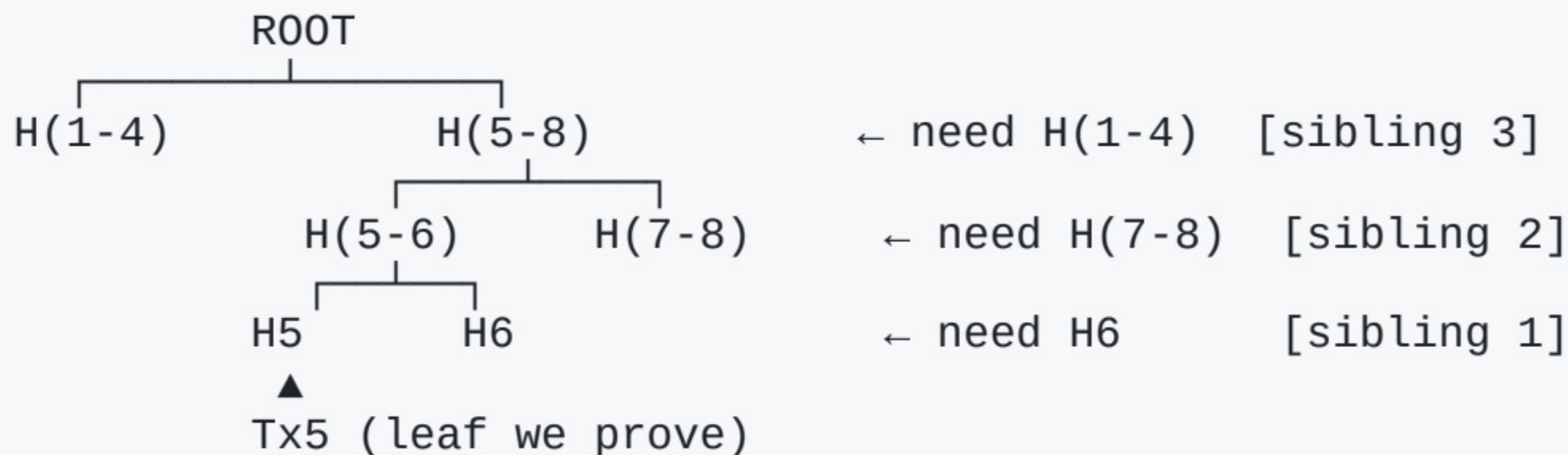
The root commits to ALL transactions — change any tx → root changes.

Gốc cam kết cho TOÀN BỘ giao dịch — sửa bất kỳ tx nào → gốc đổi.

Merkle proof by hand / Bằng chứng Merkle bằng tay

8 leaves; prove **Tx5** is in the block. Provide 3 sibling hashes:

8 lá; chứng minh Tx5 nằm trong block. Cần 3 băm anh em:



Verifier recomputes $H5 \rightarrow H(5-6) \rightarrow H(5-8) \rightarrow R00T$ and compares. **3 hashes = $\log_2 8$.**

Người kiểm tính ngược lên gốc rồi so sánh. 3 băm = $\log_2 8$.

Merkle proof — the numbers / Con số

- Proof size = **$\log_2(n)$** hashes / *kích thước proof = $\log_2(n)$ bít*
- 1,024 txs → 10 hashes = **320 bytes** (not the whole block / *không phải cả block*)
- 1,000,000 txs → **20 hashes** (~640 bytes)

n (txs)	proof hashes	~bytes
$2^{10} = 1,024$	10	320
$2^{16} = 65,536$	16	512
$2^{20} \approx 1,000,000$	20	640

Verify membership without downloading the block. *Kiểm tra cách thành viên mà không tải cả block.*

Where Merkle trees live / Merkle nằm ở đâu

- **SPV light wallets** — verify a tx with just block headers + proof
Ví nhẹ SPV — kiểm tx chỉ với header + proof
- **Ethereum state trie** — Merkle-Patricia proofs for light clients
Cây trạng thái Ethereum — bằng chứng Merkle-Patricia cho light client
- **Airdrops** — OpenZeppelin `MerkleProof`, one root = cheap allowlist of thousands
Airdrop — một gốc = allowlist rẻ cho hàng nghìn địa chỉ
- **Rollups** — commit L2 state to L1 as a root / *cam kết trạng thái L2 lên L1*
- **CEX proof-of-reserves** — Merkle tree of customer balances / *bằng chứng dự trữ sản*

Break 10' / Giải lao

Try / *thử*: andersbrownworth.com/blockchain — visual hash & block demo

Hour 2 / Giờ 2

Signatures, keys & addresses

Chữ ký, khóa & địa chỉ

ECC in 10 minutes / ECC trong 10 phút

- Curve points + "point addition"; base point **G**
Các điểm trên đường cong + phép "cộng điểm"; điểm cơ sở G
- Private key **d** = random 256-bit integer / *khóa bí mật = số nguyên 256 bit ngẫu nhiên*
- Public key **Q** = **d**·**G** — fast to compute / *tính xuôi nhanh*
- **Q** → **d** = **discrete log problem** $\approx 2^{128}$ **ops** — infeasible / *bất khả thi*

This one-way street IS the wallet security model.

Con đường một chiều này chính là mô hình an toàn của ví.

secp256k1 by the numbers / secp256k1 qua các con số

Param	Value / Giá trị
Curve / Đường cong	$y^2 = x^3 + 7$ (a=0, b=7)
Field prime p	$2^{256} - 2^{32} - 977$
Order n	$\approx 1.158 \times 10^{77}$ (prime)
Cofactor h	1
Security / An toàn	~128-bit

- **$d \in [1, n-1]$** — that's why "any 256-bit number" is a valid key
 d nằm trong $[1, n-1]$ — vì thế "gần như mọi số 256 bit" là khóa hợp lệ
- Same curve for Bitcoin **and** Ethereum EOAs (SEC2 standard).

Curves in the wild / Đường cong ngoài thực tế

Curve	Used by / Dùng bởi	Signature
secp256k1	Bitcoin, Ethereum EOA	ECDSA, Schnorr (Taproot)
ed25519	Solana, TON, Cardano	EdDSA (deterministic)
BLS12-381	Ethereum PoS validators	BLS (aggregatable / <i>gộp được</i>)

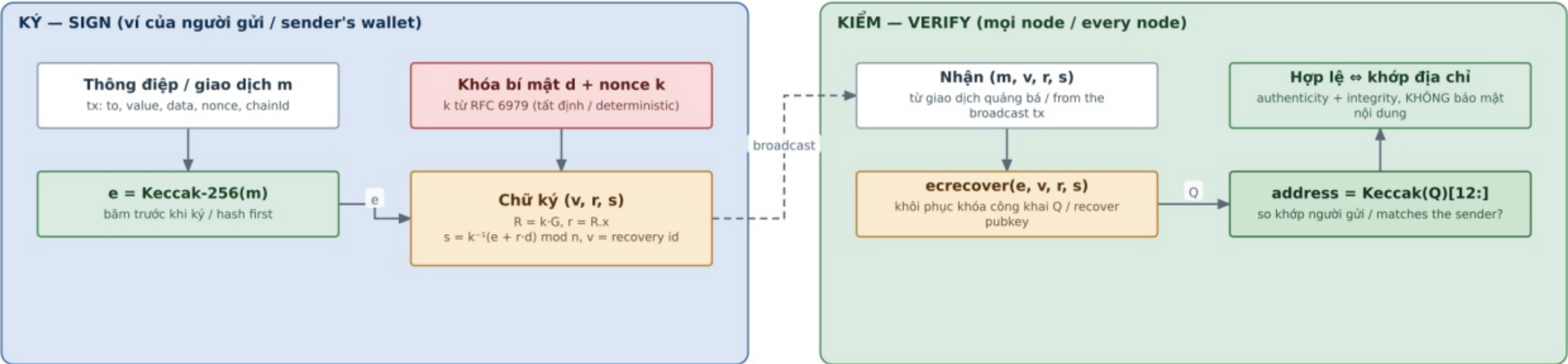
Different curve = different signature algorithm = different capabilities.

Đường cong khác → thuật toán ký khác → khả năng khác.

ECDSA: sign → verify → recover

ECDSA trên secp256k1: ký, kiểm và ecrecover — signing, verifying & address recovery

Ethereum: giao dịch KHÔNG chứa khóa công khai — địa chỉ được KHÔI PHỤC từ chữ ký • An Ethereum tx carries no public key — the address is RECOVERED from the signature



⚠ LẶP LẠI NONCE k = LỘ KHÓA BÍ MẬT — REUSED NONCE k LEAKS THE PRIVATE KEY.
Hai chữ ký cùng k ⇒ hai phương trình tuyến tính ⇒ giải ra d. Sony PS3 (2010) dùng k hằng số; nhiều ví Bitcoin RNG lỗi bị rút sạch. Two sigs sharing k ⇒ solve for d. Fix: RFC 6979.

Ứng dụng của ecrecover: meta-transaction, permit EIP-2612, Sign-in with Ethereum. Chống REPLAY: luôn đưa chainId + nonce vào dữ liệu ký (EIP-712 typed data). Uses: meta-txs, EIP-2612 permit, SIWE login. Anti-replay: include chainId + nonce (EIP-712).

ECDSA — signing, step by step / Ký từng bước

Inputs: private key **d**, message **m**, curve order **n**, generator **G**.

```
1.  e  = H(m)                # hash the message / băm thông điệp
2.  k  = random nonce in [1, n-1] # MUST be unique & secret / duy nhất & bí mật
3.  R  = k·G ; r = R.x mod n    # (if r = 0, pick new k)
4.  s  = k-1 · (e + r·d) mod n  # (if s = 0, pick new k)
5.  signature = (r, s)         # Ethereum also stores v (recovery id)
```

- **Low-s rule (EIP-2):** keep $s \leq n/2$ → blocks signature malleability

Quy tắc low-s: giữ $s \leq n/2$ → chặn tính "dẻo" của chữ ký

ECDSA — verifying, step by step / Kiểm từng bước

Inputs: public key **Q**, message **m**, signature **(r, s)**.

```
1.  check r, s in [1, n-1]           # reject out-of-range / loại giá trị sai miền
2.  e  = H(m)
3.  w  = s-1 mod n
4.  u1 = e·w mod n ; u2 = r·w mod n
5.  P  = u1·G + u2·Q                 # a curve point / một điểm trên đường cong
6.  valid ⇔ P.x mod n == r           # signature matches / chữ ký khớp
```

Verification uses **only the public key** — never d.

Kiểm chỉ dùng khóa công khai — không bao giờ cần d.

⚠ **The deadly nonce / Nonce chết người**

Reuse k in two signatures $\rightarrow$ two linear equations $\rightarrow$ **solve for private key d**

Dùng lại k trong hai chữ ký $\rightarrow$ hai phương trình tuyến tính $\rightarrow$ giải ra khóa bí mật d

- **Sony PS3 (2010):** constant $k \rightarrow$ master key extracted / k hằng số $\rightarrow$ lộ khóa chủ
- Bitcoin wallets with broken RNG $\rightarrow$ drained / ví RNG lỗi $\rightarrow$ bị rút sạch

Two signatures, same k : $s_1 - s_2 = k^{-1}(e_1 - e_2) \rightarrow$ recover $k \rightarrow$ recover d .

Hai chữ ký cùng k : giải ra k , rồi ra d .

Fix: RFC 6979 — deterministic k / Khắc phục: k tất định

- Derive k **deterministically** from (private key d, message m)
Sinh k một cách tất định từ (khóa d, thông điệp m)
- Same message → same signature (no RNG needed at sign time)
Cùng thông điệp → cùng chữ ký (không cần RNG khi ký)
- Removes the "bad randomness" foot-gun entirely
Loại bỏ hoàn toàn bẫy "ngẫu nhiên lỗi"

You'll SEE this in the lab: signing twice gives the *identical* (r, s).
Bạn sẽ THẤY ở lab: ký hai lần cho ra (r, s) y hệt.

Ethereum (v, r, s) & ecrecover

- Tx carries **no public key** — address is *recovered* from the signature
Giao dịch không chứa khóa công khai — địa chỉ được khôi phục từ chữ ký
- **v** = recovery id (which of 2 candidate points) → picks the right pubkey
v = mã khôi phục (chọn 1 trong 2 điểm) → ra đúng khóa công khai
- Powers / *nền tảng của*: meta-transactions • EIP-2612 `permit` • Sign-in with Ethereum
- **Anti-replay / chống replay**: include `chainId` + nonce in signed data → **EIP-712**

Where signatures live / Chữ ký nằm ở đâu

- **Every transaction** — proves the sender authorized it / *mọi giao dịch — chứng minh người gửi cho phép*
- **ecrecover in Solidity** — verify off-chain sigs on-chain / *kiểm chữ ký ngoài chuỗi ngay trong hợp đồng*
- **Gasless UX** — meta-tx & **permit**: user signs, relayer pays gas / *người dùng ký, relayer trả gas*
- **Login** — Sign-in with Ethereum (SIWE), no password / *đăng nhập không mật khẩu*
- **Validators (PoS)** — attestations signed with BLS / *chứng thực ký bằng BLS*

Signature = the "yes, I authorize this" of the whole system.

Chữ ký = tiếng "vâng, tôi cho phép" của cả hệ thống.

Schnorr • BLS • EdDSA — one line each

Scheme	Superpower / Siêu năng lực	Where
Schnorr	aggregation: multisig looks like single-sig / <i>đa chữ ký trông như đơn</i>	Bitcoin Taproot
BLS	1000s of sigs → one — makes 1M validators feasible / <i>gộp nghìn thành một</i>	Ethereum PoS
EdDSA	deterministic by design — no nonce foot-gun / <i>tất định, hết bấy nonce</i>	Solana, TON

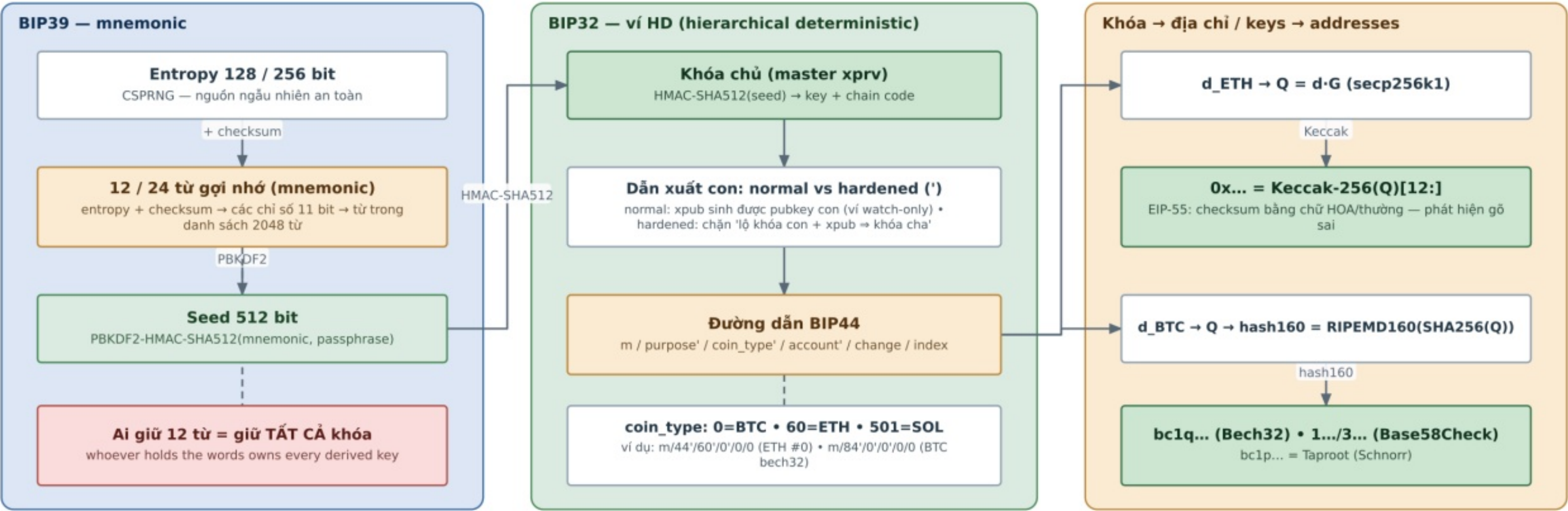
Aggregation = smaller blocks + more privacy + feasible mega-validator sets.

Gộp chữ ký = block nhỏ hơn + riêng tư hơn + tập validator khổng lồ khả thi.

From entropy to address / Từ entropy đến địa chỉ

Từ entropy đến địa chỉ: BIP39 → BIP32 → BIP44 — From entropy to address

Một mnemonic → mọi coin, mọi tài khoản • One mnemonic → every coin, every account



Đây chính là điều đã làm ở Buổi 1 khi 'Backup Root Key' trên TrustKeys/MetaMask: app sinh entropy, hiện 12 từ, và từ đó dẫn xuất mọi địa chỉ các em thấy trong ví. This is exactly what 'Backup Root Key' did in Session 1: the app generated entropy, showed 12 words, and derived every address you saw in the wallet.

BIP39 — mnemonic

1. Entropy 128/256 bit (CSPRNG)
2. ◦ checksum → 11-bit indices → **12/24 words** (fixed 2048-word list / *danh sách 2048 từ cố định*)
3. PBKDF2(mnemonic, passphrase) → **seed 512 bit**
 - 128-bit entropy → 12 words • 256-bit → 24 words (*mỗi từ mã hóa 11 bit*)
 - Optional **passphrase** = "25th word" → a different wallet (*passphrase* = "*từ thứ 25*")

Whoever holds the words holds ALL the keys — Session 1's "Backup Root Key".

Ai giữ 12 từ = giữ TẤT CẢ khóa.

BIP32 — the HD tree / Cây HD

- Seed → **master key** (xprv) → child keys via **HMAC-SHA512**
Seed → khóa chủ → khóa con bằng HMAC-SHA512
- One seed → a whole **tree** of keys, all recoverable from the 12 words
Một seed → cả cây khóa, đều khôi phục được từ 12 từ
- **xpub** (extended public key) can derive child *public* keys → **watch-only** wallets
xpub sinh được khóa công khai con → ví chỉ-xem (không tiêu được)

This is why one backup → 50 addresses in your wallet app.

Đây là lý do một lần backup → app ví hiện ra 50 địa chỉ.

Hardened vs normal derivation / Dẫn xuất hardened vs thường

	Normal	Hardened (')
Needs / Cần	parent pubkey	parent privkey
xpub can derive child pubkeys?	✅ yes	❌ no
Use / Dùng	watch-only, change , index	purpose' , coin' , account'

⚠️ **Normal derivation risk:** leaked child *private* key + parent xpub $\Rightarrow$ parent private key.

Rủi ro dẫn xuất thường: lộ khóa con + xpub cha $\Rightarrow$ suy ra khóa cha.

Hardened blocks exactly this $\rightarrow$ used for the top path levels.

Hardened chặn đúng điều đó $\rightarrow$ dùng cho các tầng trên của đường dẫn.

BIP44 — path anatomy / Giải phẫu đường dẫn

m / purpose' / coin_type' / account' / change / index

Segment	Example	Meaning / Ý nghĩa
purpose'	44'	BIP44 (49'/84'/86' = other addr types)
coin_type'	60'	0'=BTC, 60'=ETH, 501'=SOL (SLIP-44)
account'	0'	user account / tài khoản người dùng
change	0	0 = receive, 1 = change / nhận / thối
index	0	address # 0, 1, 2... / địa chỉ thứ...

m/44'/60'/0'/0/0 = **first Ethereum account** (the ' = hardened, +2³¹).
= tài khoản Ethereum đầu tiên. Dấu ' = hardened.

Ethereum addresses & EIP-55 / Địa chỉ Ethereum

- **Address** = `0x` + last 20 bytes of **Keccak-256(pubkey)**
Địa chỉ = 0x + 20 byte cuối của Keccak-256(khóa công khai)
- **EIP-55**: mixed UPPER/lower-case = a checksum built from the hash
EIP-55: hoa/thường trộn = checksum tính từ chính giá trị băm
- Wallets **reject** a mistyped address whose case doesn't match
Ví từ chối địa chỉ gõ sai khi hoa/thường không khớp

0xAb5801a7D398351b8bE11C439e05C5B3259aEC9B
0xab5801a7d398351b8be11c439e05c5b3259aec9b

✓ valid checksum
⚠ all-lowercase (no check)

Bitcoin addresses / Địa chỉ Bitcoin

- pubkey → **SHA-256** → **RIPEMD-160** ("hash160") → encode
khóa công khai → SHA-256 → RIPEMD-160 → mã hóa

Prefix	Format	Type
1...	Base58Check	legacy P2PKH
3...	Base58Check	P2SH (multisig, segwit-wrap)
bc1q...	Bech32	native SegWit
bc1p...	Bech32m	Taproot

Encoding carries a built-in checksum → mistyped address = invalid.

Cách mã hóa có sẵn checksum → gõ sai = không hợp lệ.

⚠ **Address security / An toàn địa chỉ**

- **Address poisoning:** scammers seed your history with a look-alike whose first/last 4 chars match → you copy the wrong one
"Đầu độc địa chỉ": kẻ gian gieo địa chỉ giống 4 ký tự đầu/cuối → bạn copy nhầm
- **Clipboard hijack malware:** silently swaps a copied address
Mã độc chiếm clipboard: tráo địa chỉ vừa copy

Defense / Phòng vệ: check the **middle** characters, not just the ends; verify on the device screen.

Soi phần GIỮA, không chỉ hai đầu; đối chiếu trên màn hình thiết bị.

Common misconceptions / Ngộ nhận thường gặp

Misconception / Ngộ nhận	Correction / Chỉnh lại
"Hashing = encryption"	one-way, keyless, not reversible
"Signature hides the message"	gives authenticity + integrity, not secrecy
"Coins live in the wallet app"	chain stores balances; wallet stores keys
"12 words are on a server"	generated & kept locally
"Ethereum uses NIST SHA3"	it's Keccak-256 (different padding)



Break 10' → Lab

```
pip install eth-account
```

Hour 3 / Giờ 3

Lab — Merkle + sign/verify/recover

Thực hành

Lab (50') — pass = CHECKs OK + demo 3.3

'	Lab	Deliverable / Sản phẩm
10	3.1 Hash + toy PoW	Q1-Q2: work $\times 16$ per zero? verify cost?
20	3.2 Merkle tree	3 TODOs $\rightarrow$ all <code>CHECK OK</code> ; Q3-Q4
15	3.3 Sign & recover	same sig twice (why?); tampered msg $\rightarrow$ different address
5	Wrap-up	push code + <code>answers.md</code> before 23:59

Files / tệp: `lab/lab03_worksheet.md` , `lab/merkle_starter.py`

Lab 3.3 — what you'll observe / Điều bạn sẽ thấy

```
from eth_account import Account
from eth_account.messages import encode_defunct

acct = Account.create() # NEVER a real key / KHÔNG dùng khóa thật
msg = encode_defunct(text="I attended Session 3")
sig = Account.sign_message(msg, acct.key)
who = Account.recover_message(msg, signature=sig.signature)
print(who == acct.address) # True – recovered from sig alone
```

- Sign twice → **identical** (r, s) → that's **RFC 6979** / *ký hai lần y hệt* → *RFC 6979*
- Tamper 1 char → **different** recovered address → integrity proven / *sửa 1 ký tự* → *địa chỉ khác*

Homework / Bài về nhà (trước Buổi 4)

1. Merkle proof sizes for $n = 2^{10}, 2^{16}, 2^{20}$ — plot / *đo & vẽ đồ thị*
2. iancoleman.io/bip39 **OFFLINE only** — verify `m/44'/60'/0'/0/0` = MetaMask address #0; write $\frac{1}{2}$ page on the risks / *kiểm chứng đường dẫn + nửa trang về rủi ro*
3. Read / *đọc*: Mastering Bitcoin ch.4 • Mastering Ethereum ch.4–6

Next / Buổi sau

Session 4 — Bitcoin In Depth & Consensus

UTXO • Script • Mining economics • Nakamoto consensus

Hashes + signatures **assemble into Bitcoin** — bring your laptop!

Băm + chữ ký lắp ráp thành Bitcoin — nhớ mang laptop!

References / Tài liệu tham khảo

- **NIST FIPS 180-4** — Secure Hash Standard (SHA-2) • **FIPS 202** — SHA-3/Keccak
csrc.nist.gov/publications
- **SEC 2** — Recommended Elliptic Curve Domain Parameters (secp256k1) — secg.org
- **RFC 6979** — Deterministic ECDSA (nonce k) — datatracker.ietf.org/doc/html/rfc6979
- **BIP-32** (HD wallets) • **BIP-39** (mnemonic) • **BIP-44** (paths) • **SLIP-44** (coin types)
github.com/bitcoin/bips
- **EIP-55** (address checksum) • **EIP-712** (typed signing) • **EIP-2612** (permit) —
eips.ethereum.org
- **Mastering Bitcoin** ch.4 (Keys & Addresses) • **Mastering Ethereum** ch.4–6 —
github.com/bitcoinbook, [/ethereumbook](https://github.com/ethereumbook)